

INSIGHTX · PROJECT INTRO · 2026

讀完所有評論， 再給你一份報告。

輸入一個 Google Maps 或 YouTube 連結 — AI 自動整理成情緒分析、SWOT、回覆草稿、行銷貼文、週行動計畫。

顧客每天在留言，但意見很少變成決策。

管理者每天打開後台看到一堆五星評論，但下一步要做什麼？沒人說得出來。

太碎了

意見散在 Google Maps、YouTube、IG、LINE 各處，沒有人有時間全部讀完。

太多了

50 則留言已經是負擔，500 則直接放棄。看得到「很多人說好吃」，看不到「冷氣太強客人不想久坐」這種真正的細節。

回應壓力大

負評上線，店長被情緒帶著走，回覆要嘛太硬要嘛太軟，反而失去挽回客人的機會。

新店長沒練習場

剛升上店長的人沒處理過真正的危機，學費全部是用真實客人賠的。

一個 URL 換一份完整報告。

把「讀評論 → 想策略 → 寫回覆 → 練手」串成同一條流水線。

跨平台抓料，不用開瀏覽器

Google Maps 用 Serper API 抓店家評論，YouTube 用官方 Data API 抓影片留言。
速度快、也不容易被反爬擋掉。

9 個 AI 功能共用一份資料

情緒分析、SWOT、回覆草稿、行銷文案、根源分析、週計畫、培訓劇本、內部信、AI 對話顧問。
依平台切換語氣（餐廳 / 零售 / YouTuber）。

多店家工作區

每位使用者有自己的工作區，可以新增多個店家、保留完整歷史紀錄。
隨時回看上個月對某家分店的分析。

管理者決策模擬遊戲

把真實負評倒進小遊戲，AI 當你的虛擬顧問即時給回饋。
練手不用拿真客人試刀。

9 個 AI 功能，共用一份原始評論。

依平台切換語氣——餐廳老闆看到的是門市建議，YouTuber 看到的是頻道策略。

01

情緒分析

正負向 + 主題分布

02

SWOT

自動生成戰略矩陣

03

回覆草稿

每則負評一個對應草稿

04

行銷文案

門市活動 / 影片宣傳

05

根源分析

找出真正的痛點

06

週行動計畫

一週要做哪些具體事項

07

培訓劇本

員工 / 剪輯師訓練教材

08

內部信

門市 / 團隊週報

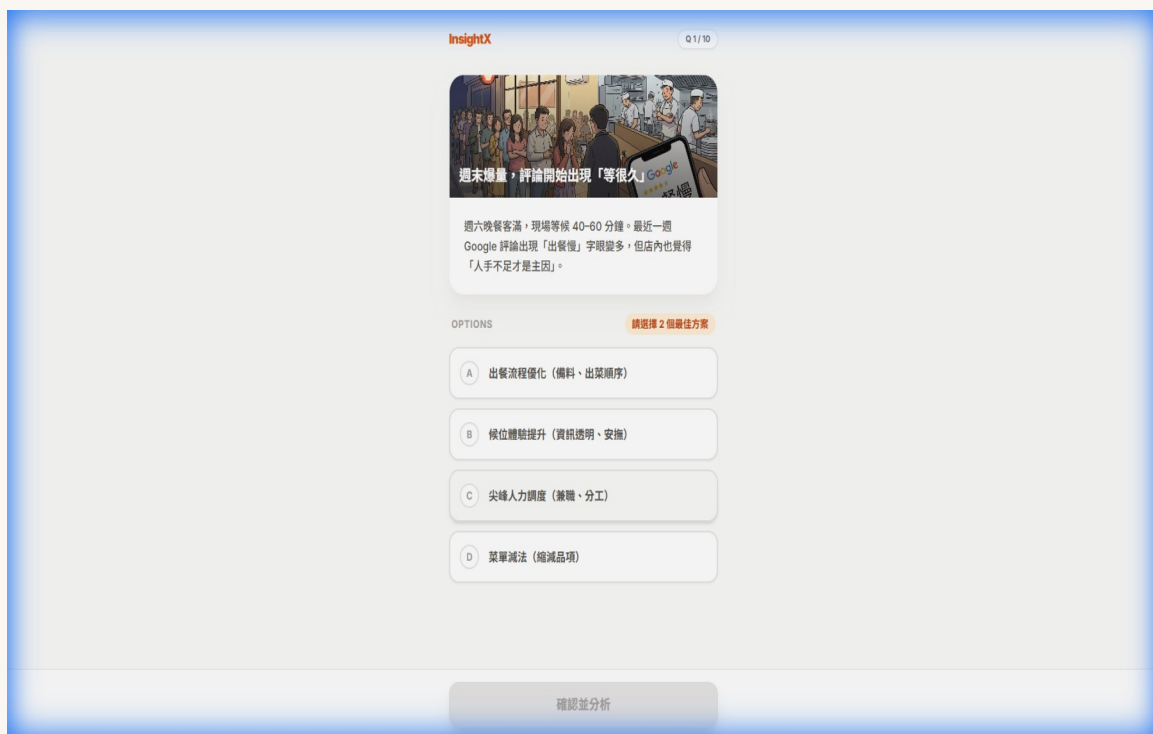
09

AI 顧問

隨時可問的虛擬導師

用真實負評，練決策。

新任店長最痛的事——第一次處理客訴。這個遊戲讓你在真實負評上練手。



1. AI 出題

把真實負評變成情境問句

2. 你選回應

從多個策略選一個你會做的

3. AI 給回饋

評估你的選擇，給情商分數 + 建議

—— 學費，不用拿真客人賠。

3 個決定，影響使用體驗。

不堆技術名詞 — 講為什麼這樣做、解了什麼問題。

● 多人也能安全使用

最初版本所有人共用一個帳號，任何人打開都看到別人的資料。
現在每位訪客一進站，自動拿到一個身分（用 cookie 記下來），
所有資料綁在這身分上。沒有註冊、沒有密碼，但每個人的工作區完全分開。

● 不會因為 AI 抖一下就掛

免費版 Gemini 半夜常常會卡。我做了一個自動換模型的機制：
主用快的，失敗就降到大的，再不行換 Google 的旗艦，最後 lite 版兜底。
同一次請求一路 fallback，使用者完全感覺不到。

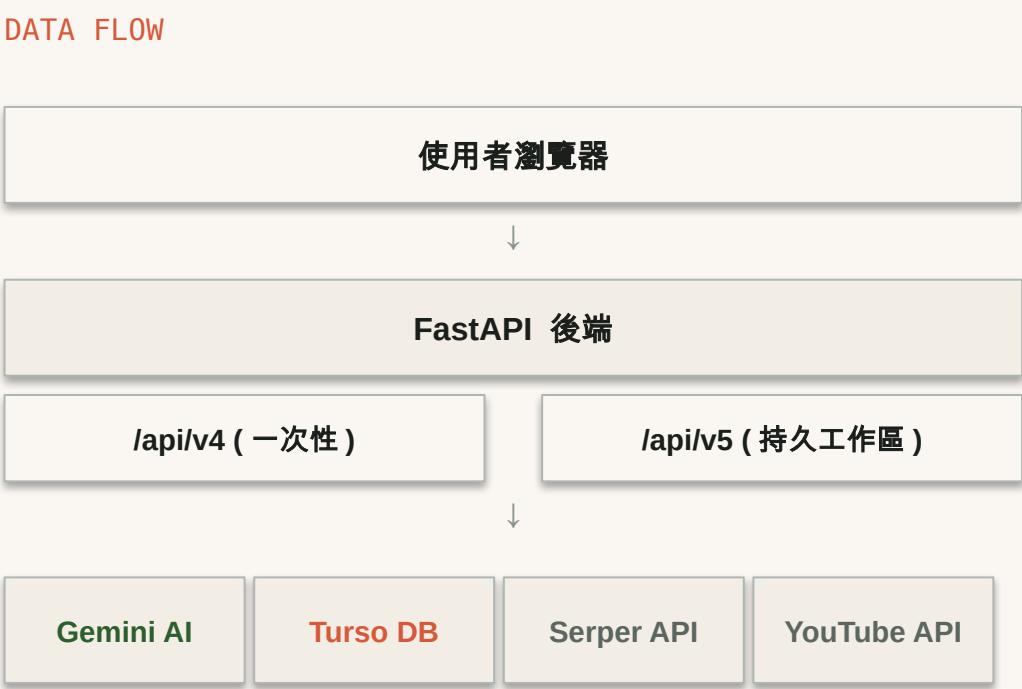
● 雙 AI 寫 code

改動比較大的時候，我習慣讓 Codex（另一個 AI 助手）幫我看一遍程式碼。
一個 AI 寫、另一個 AI 挑毛病。抓出來的問題比自己看更多 —
等於多了一個免費的 reviewer。

從一張圖看整個系統。

後端 FastAPI ， AI 是 Gemini ， 部署在 Hugging Face Spaces — 全部用免費方案組起來。

TECH STACK	
後端	FastAPI · Python 3.10+
資料庫	Turso · SQLite (libsql)
AI	Google Gemini (多模型 fallback)
爬蟲	Serper API + YouTube Data API
前端	React 18 + Tailwind CSS
部署	Docker on HF Spaces (\$0 / 月)



做這個專案學到的事。

從第一版 demo 到能上線給人用的版本，學到幾件之前沒想過的事。

01

早期選錯路徑代價很大

最初用 Playwright 自動瀏覽器抓 Google Maps，後來 Google 改了反爬機制只好整段砍掉換成 Serper API。如果一開始先試 API，會省下大概兩個月。

02

API 抖動比 prompt 寫不好還麻煩

免費版 Gemini 半夜會 500，再怎麼調 prompt 都救不了。做了多模型 fallback 之後，可用率從大概 85% 拉到 99%。

03

同步反而比較好 debug

原本後端是 async 寫的，後來為了搭 Turso 改回同步。原本以為是退步，結果發現用 threading 反而出問題比較好追，效能也沒掉。

04

多一個 AI 幫忙看 code 真的有差

自己寫自己看很容易漏掉邊角，找 Codex 一起 review 後抓到不少會炸的 bug —— 多花一點時間，少踩很多坑。

現在就試。

免註冊。 免安裝。月費 \$0 。

Demo 跑在 Hugging Face Spaces 免費方案。48 小時沒人用會睡眠，第一次點要等 30 秒醒過來。

 線上 Demo

[Jordan711-insightx-demo.hf.space](https://jordan711-insightx-demo.hf.space)

 GitHub

github.com/GKS711/InsightX